

Custom workflows

In Dynamics 365, a **custom workflow plugin** is essentially a custom piece of code that extends the functionality of the platform's built-in workflows. Instead of relying solely on the standard processes provided by Dynamics 365, organizations or developers can create these plugins to add their own logic or actions to the workflow stages.

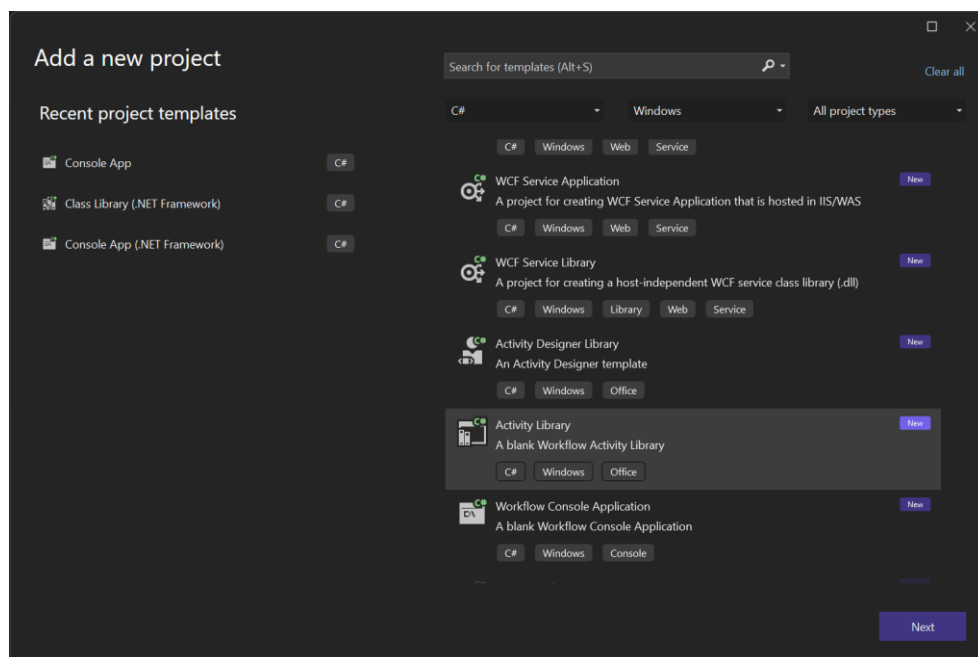
These plugins are typically written in .NET languages like C# and are designed to integrate with the Dynamics 365 workflow engine. By doing so, they can perform specialized tasks, connect with external systems, or automate processes in a way that aligns with the unique needs of the organization.

Custom workflow plugins are event-driven, meaning they are triggered by specific events such as record creation or updates, allowing for a tailored and responsive approach to business processes.

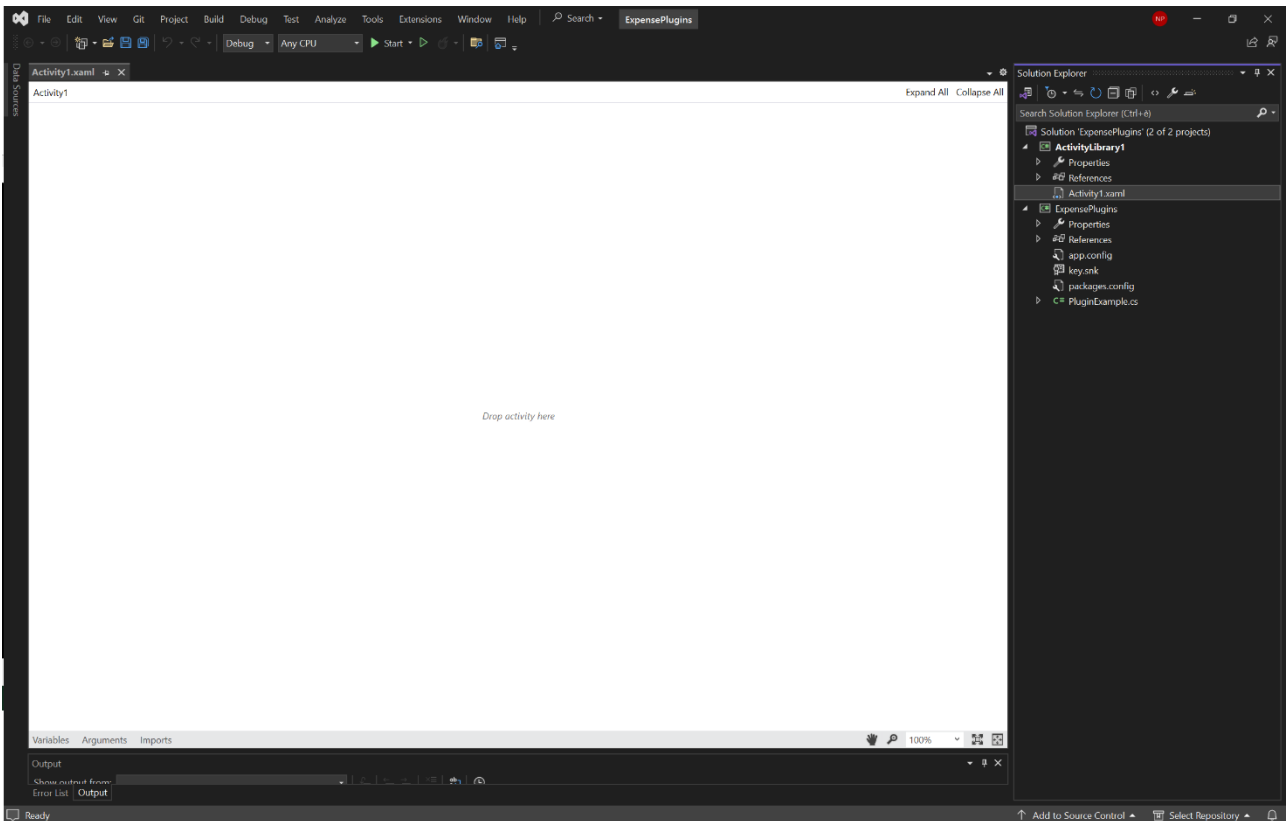
Essentially, custom workflow plugins provide a means to customize and enhance the automation capabilities of Dynamics 365, making it more adaptable to the specific requirements of a given business or industry.

1. Creating a custom workflow in Visual Studio

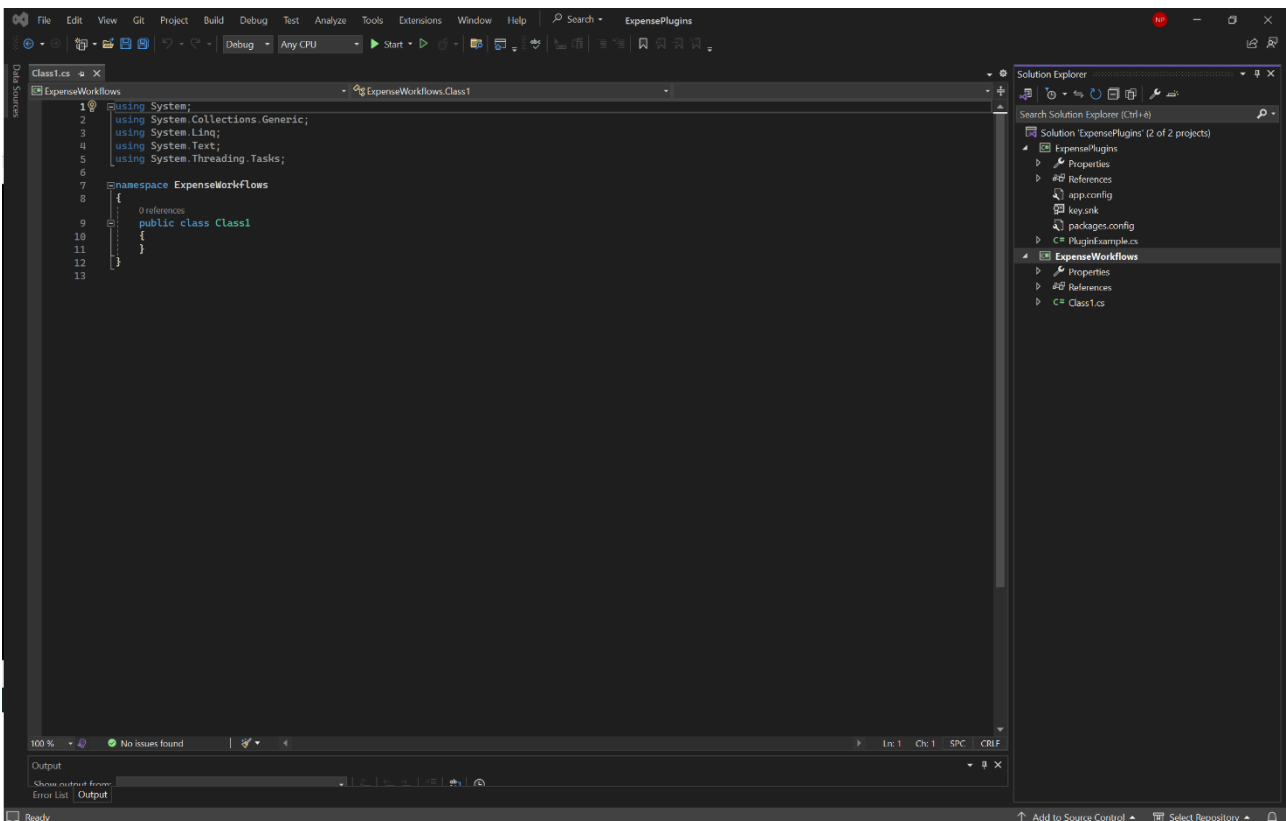
Let us create a new project in Visual Studio of type Activity Library. Notice that this requires having installed the Windows Workflow Foundation component in Visual Studio.



A .xaml file will be created. However, such file must be deleted and replaced by a class file.



Alternatively, instead of creating an Activity Library and doing the operations above, we can directly choose a Class Library project and name it ExpenseWorkflows.



We also install the Microsoft.PowerPlatform.Dataverse.Client and Microsoft.Xrm.Sdk packages.

Then we navigate to <https://learn.microsoft.com/en-us/power-apps/developer/data-platform/workflow/tutorial-create-workflow-extension> to get the documentation about creating custom workflows.

Let us register the following plugin (remember that it must be signed).

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Activities;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Workflow;
using Microsoft.Extensions.DependencyInjection;
using System.Threading;

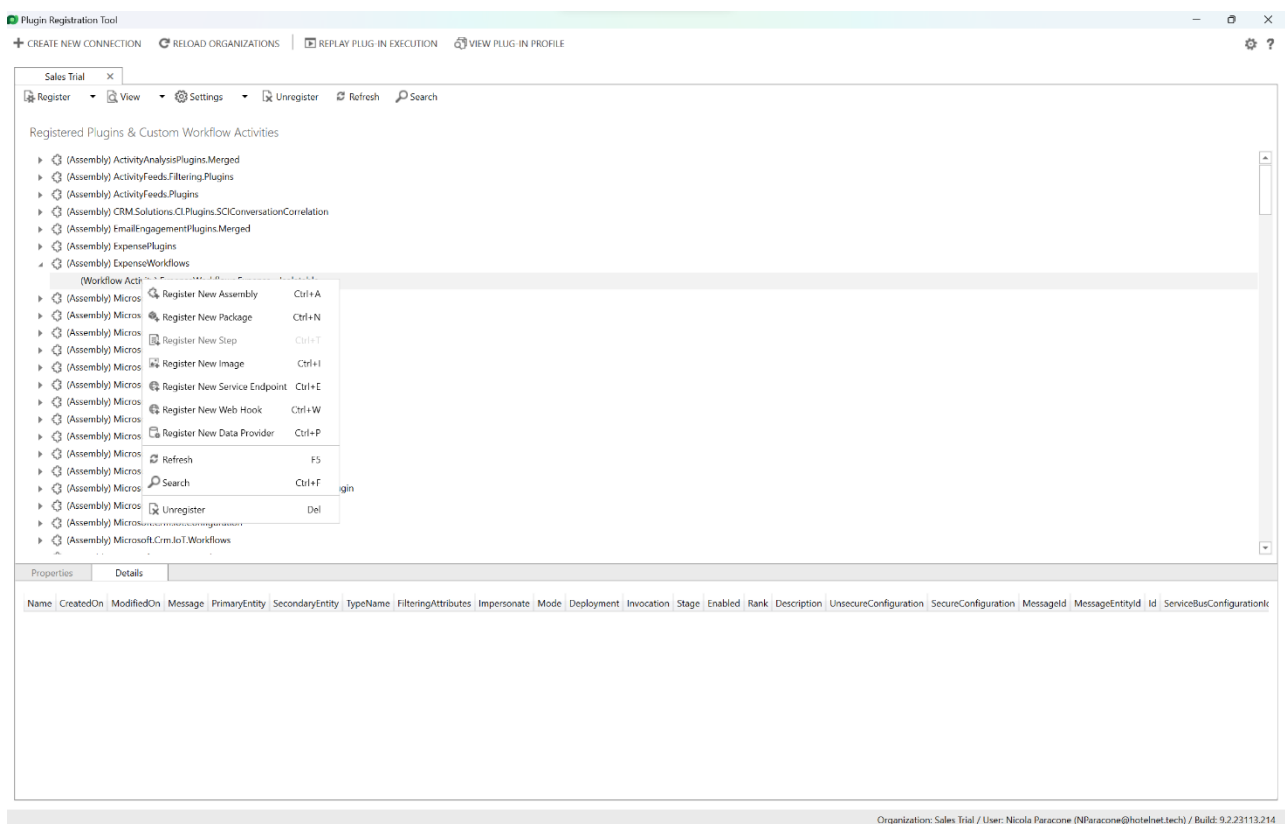
namespace ExpenseWorkflows
{
    0 references
    public class Expense : CodeActivity
    {
        0 references
        protected override void Execute(CodeActivityContext executionContext)
        {
            // Obtain the tracing service
            ITracingService tracingService = executionContext.GetExtension<ITracingService>();

            IWorkflowContext context = executionContext.GetExtension<IWorkflowContext>();
            IOrganizationServiceFactory serviceFactory = executionContext.GetExtension<IOrganizationServiceFactory>();
            IOrganizationService service = serviceFactory.CreateOrganizationService(context.InitiatingUserId);

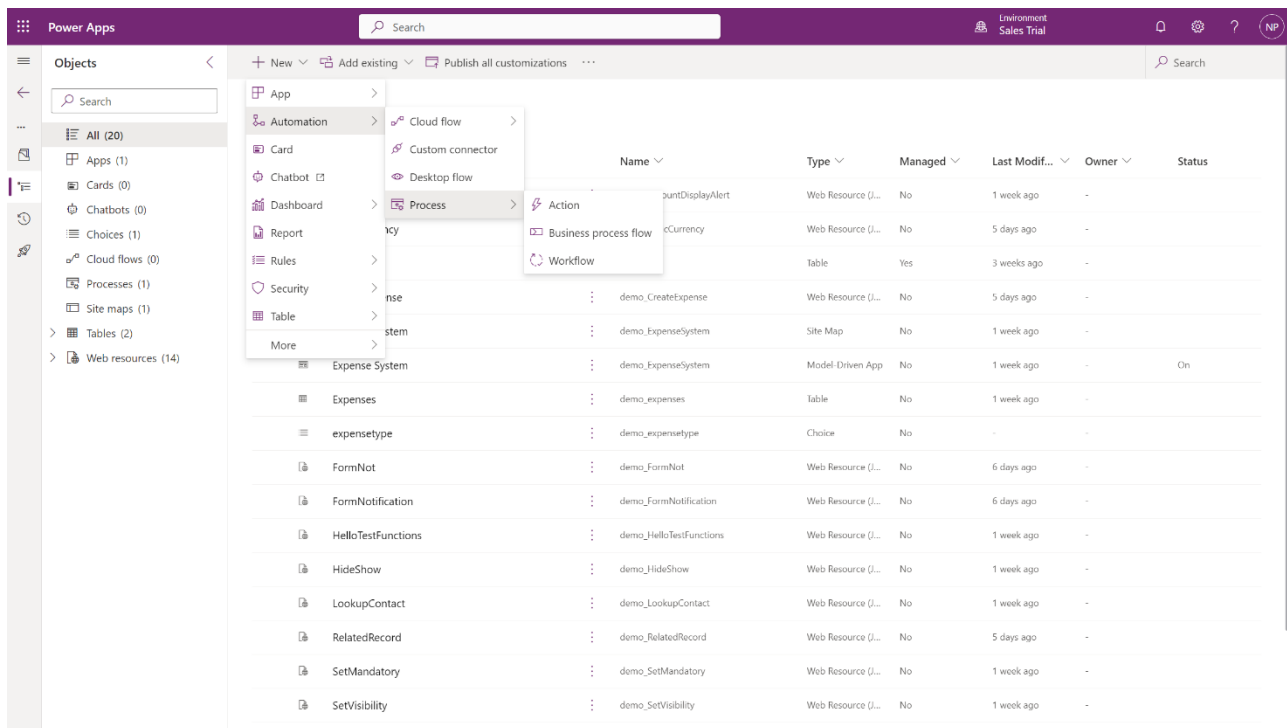
            Guid expenseid = context.PrimaryEntityId;

            tracingService.Trace(expenseid.ToString());
        }
    }
}
```

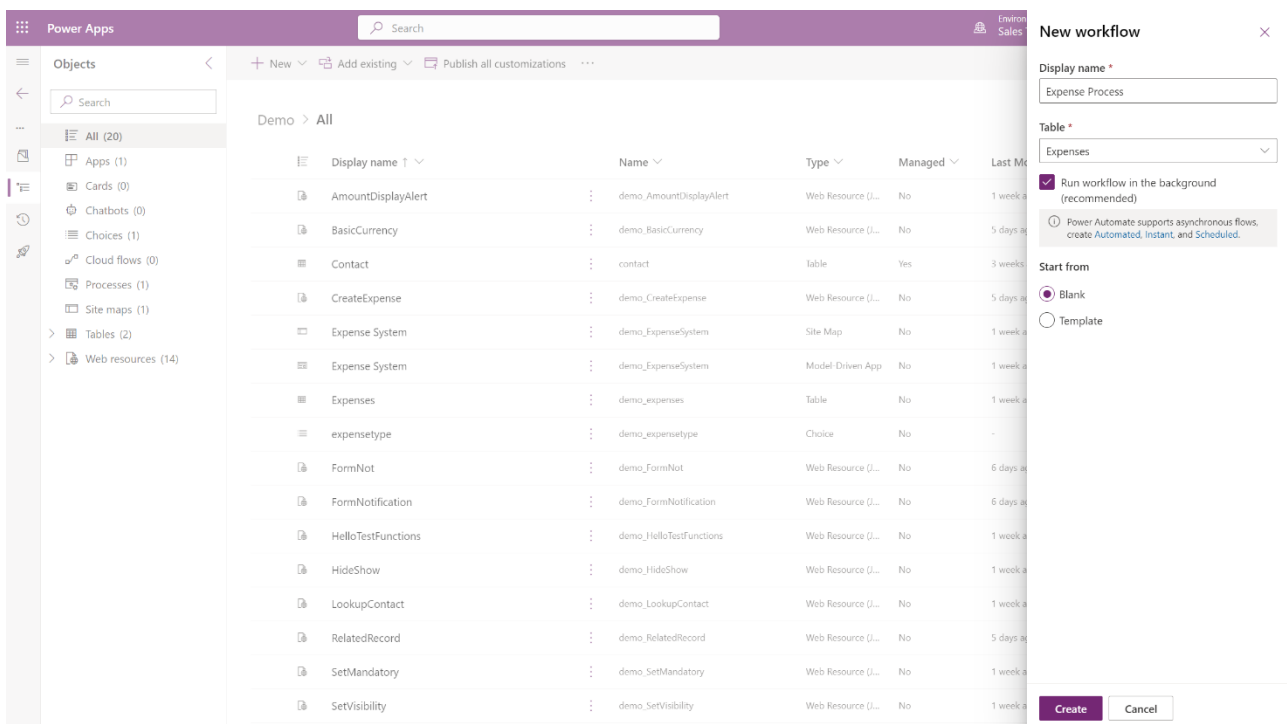
Since this is a custom workflow, there is no concept of adding steps.



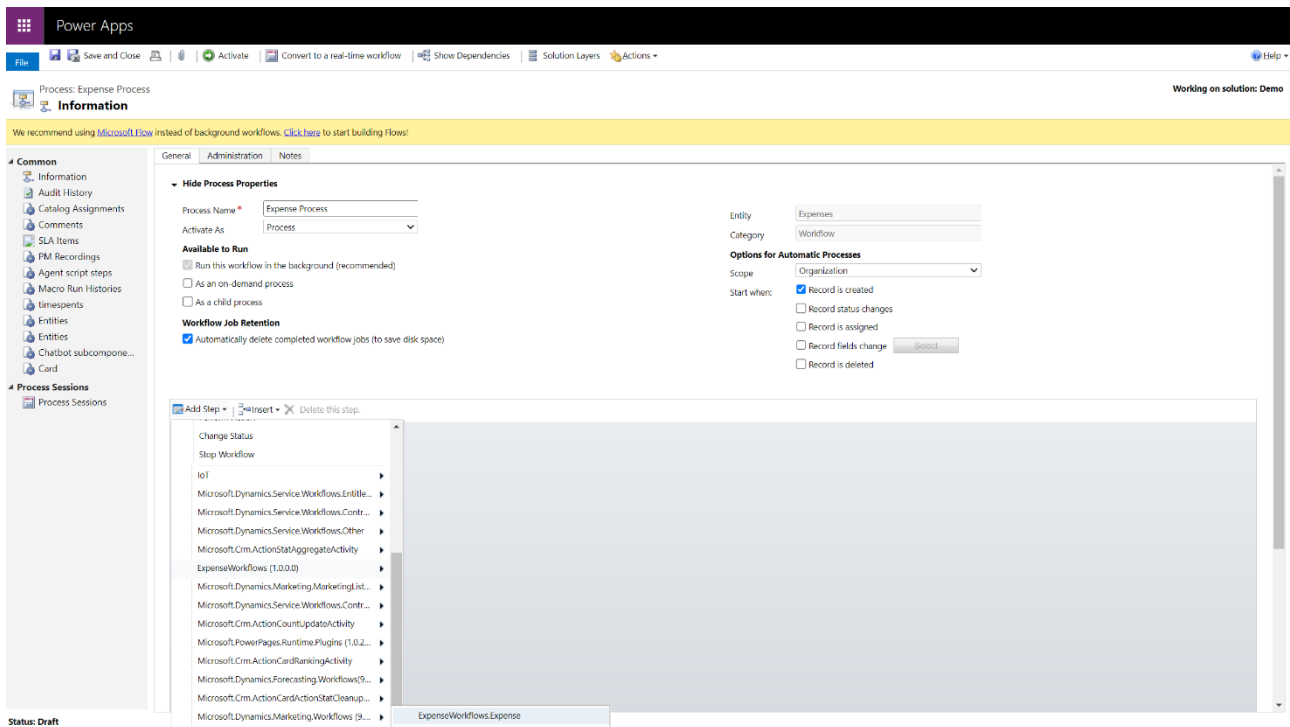
Go to Demo > All in Power Apps. Click on + New and choose Automation -> Process -> Workflow.



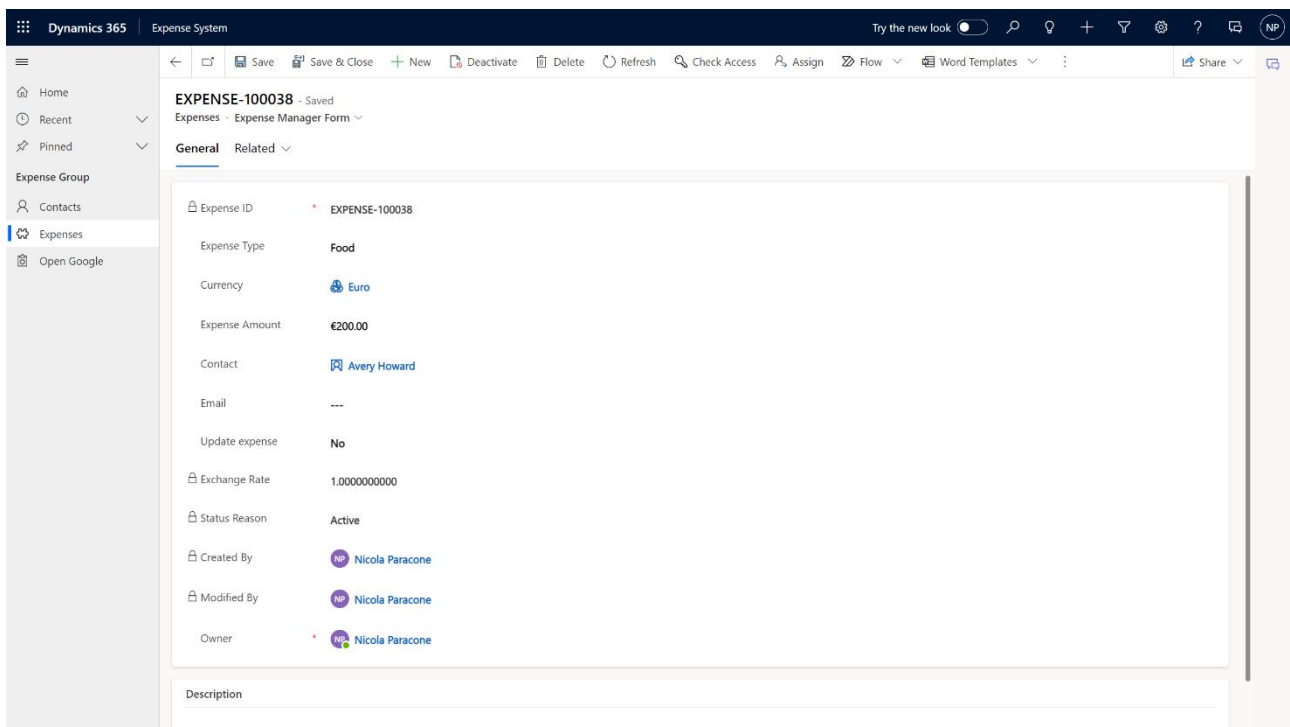
Give a name to the workflow, for example Expense Process.



Hence, when a new expense is created, we want our custom workflow to run.



Save and activate the process. Next, we create a new expense:



and check the trace log:

The screenshot shows the Dynamics 365 Plug-In Trace Log interface. The top navigation bar includes 'Dynamics 365', 'Settings', and 'Plug-In Trace Log'. The main content area is titled 'PLUG-IN TRACE LOG : INFORMATION' and displays details for a workflow activity named 'ExpenseWorkflows.Expense, ExpenseWorkflows, Versio...'. The interface is divided into several sections: 'Context' and 'Execution'. The 'Context' section shows 'Depth' as 1, 'Mode' as Asynchronous, 'Correlation Id' as 4cc783f0-6c51-4928-8373-37bb2d08e92a, and 'Request Id' as c898979b-c682-4165-b7b8-498a8c22b6df. The 'Execution' section shows 'Performance' details, including 'Execution Start Time' as 1/9/2024 10:50 AM, 'Execution Duration (ms)' as 656, and 'Message Block' as d7c3c780-d4ae-ee11-a569-000d3adeefce. The 'Exception Details' section is currently empty.

Here we will find the Guid of the expense we just created.

2. Input and output arguments

Navigate to <https://learn.microsoft.com/en-us/power-apps/developer/data-platform/workflow/workflow-extensions> to learn about workflow extensions.

In the context of custom workflows for Dynamics 365, **input parameters** are used to bring data from the triggering workflow into the custom code. These parameters are defined in the code of the custom workflow activity and allow for customization of the activity's behavior based on the values provided by the workflow.

For example, we might define an input parameter to specify the name of the entity on which the workflow operates. This information is supplied by the workflow when it calls the custom activity.

```
[Input("Entity Name")]
[Default("account")]
public InArgument<string> EntityName { get; set; }
```

On the other hand, **output parameters** are used to send data or results generated within the custom activity back to the workflow. They provide a way for the custom code to communicate information, such as a record Id or a computed value, back to the workflow.

```
[Output("Record ID")]
public OutArgument<Guid> RecordId { get; set; }
```

In summary, input parameters bring data into the custom activity from the workflow, while output parameters allow the custom activity to send results or data back to the workflow. This mechanism enables a dynamic and customizable interaction between workflows and custom code within Dynamics 365.

Consider the following code.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Activities;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Workflow;
using Microsoft.Extensions.DependencyInjection;
using System.Threading;
using Microsoft.Xrm.Sdk.Query;

namespace ExpenseWorkflows
{
    0 references
    public class Expense : CodeActivity
    {
        [Input("Contact ID")]
        [ReferenceTarget("contact")]
        1 reference
        public InArgument<EntityReference> contactRecord { get; set; }

        [Output("Fax Output")]
        1 reference
        public OutArgument<string> faxUpdate { get; set; }
        0 references
        protected override void Execute(CodeActivityContext executionContext)
        {
            // Obtain the tracing service
            ITracingService tracingService = executionContext.GetExtension<ITracingService>();

            IWorkflowContext context = executionContext.GetExtension<IWorkflowContext>();
            IOrganizationServiceFactory serviceFactory = executionContext.GetExtension<IOrganizationServiceFactory>();
            IOrganizationService service = serviceFactory.CreateOrganizationService(context.InitiatingUserId);

            Guid expenseid = context.PrimaryEntityId;

            EntityReference contactId = contactRecord.Get(executionContext);

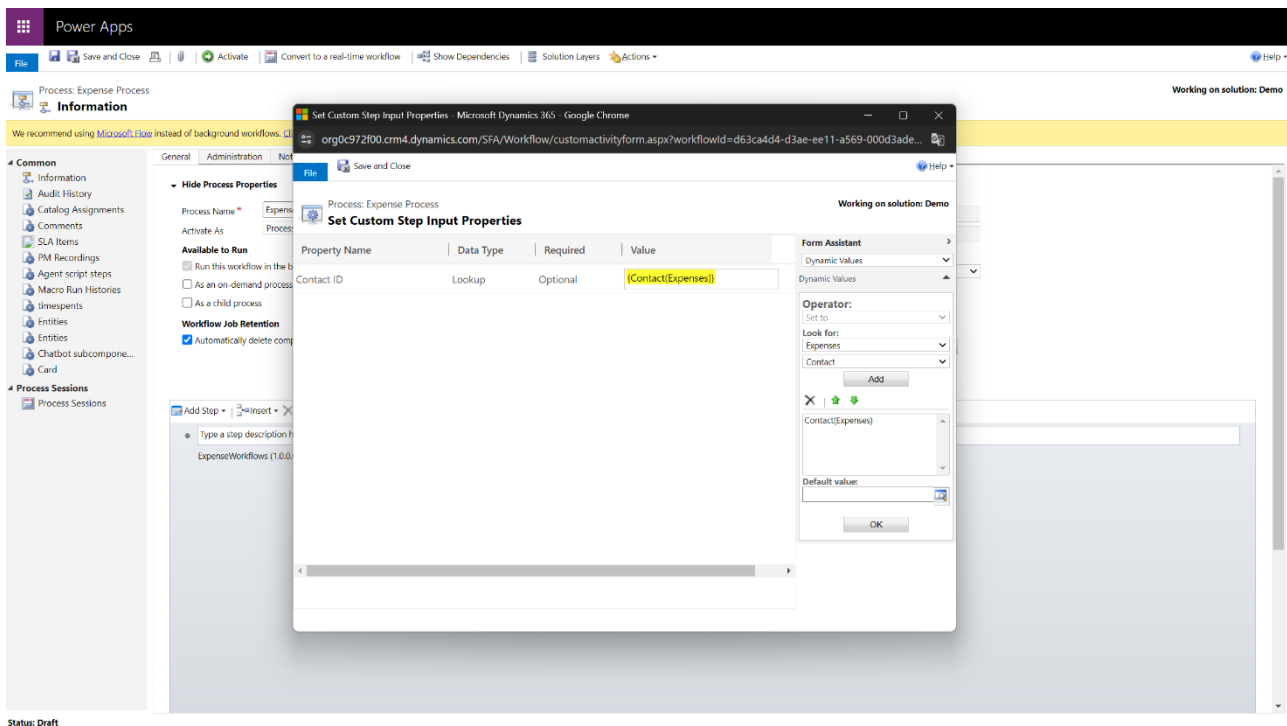
            QueryExpression query1 = new QueryExpression("demo_expenses");
            query1.ColumnSet = new ColumnSet("demo_expenseamount");
            query1.Criteria.AddCondition("demo_contact", ConditionOperator.Equal, contactId.Id);

            decimal amount = 0;
            EntityCollection ec = service.RetrieveMultiple(query1);
            foreach (Entity item in ec.Entities)
            {
                if (item.Attributes.Contains("demo_expenseamount") && item.Attributes["demo_expenseamount"] != null)
                {
                    Money expensevalue = (Money)item.Attributes["demo_expenseamount"];
                    amount = amount + expensevalue.Value;
                    tracingService.Trace(expensevalue.Value.ToString());
                }
            }

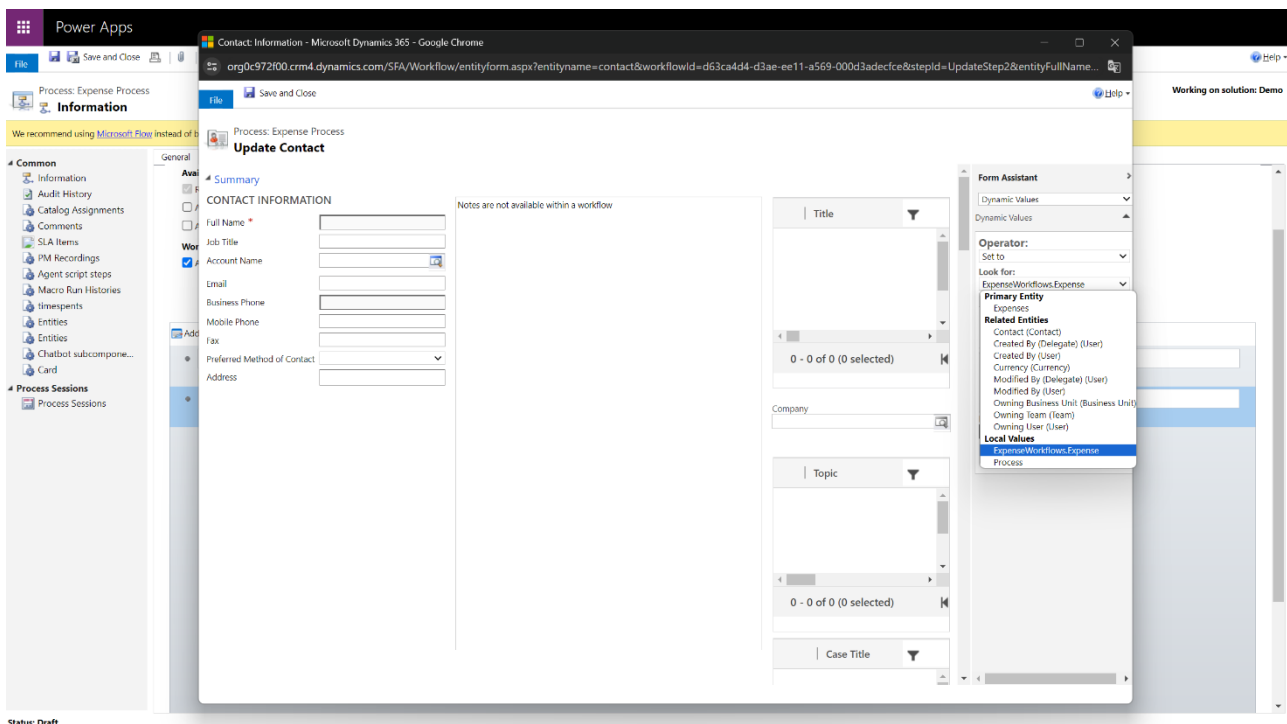
            faxUpdate.Set(executionContext, "Expense amount is: " + amount.ToString());
        }
    }
}
```

Update the plugin, deactivate the process in Power Apps, and refresh the workflow designer page.

Click on Set Properties. Set Contact(Expenses) as Value and the click on Save and Close.

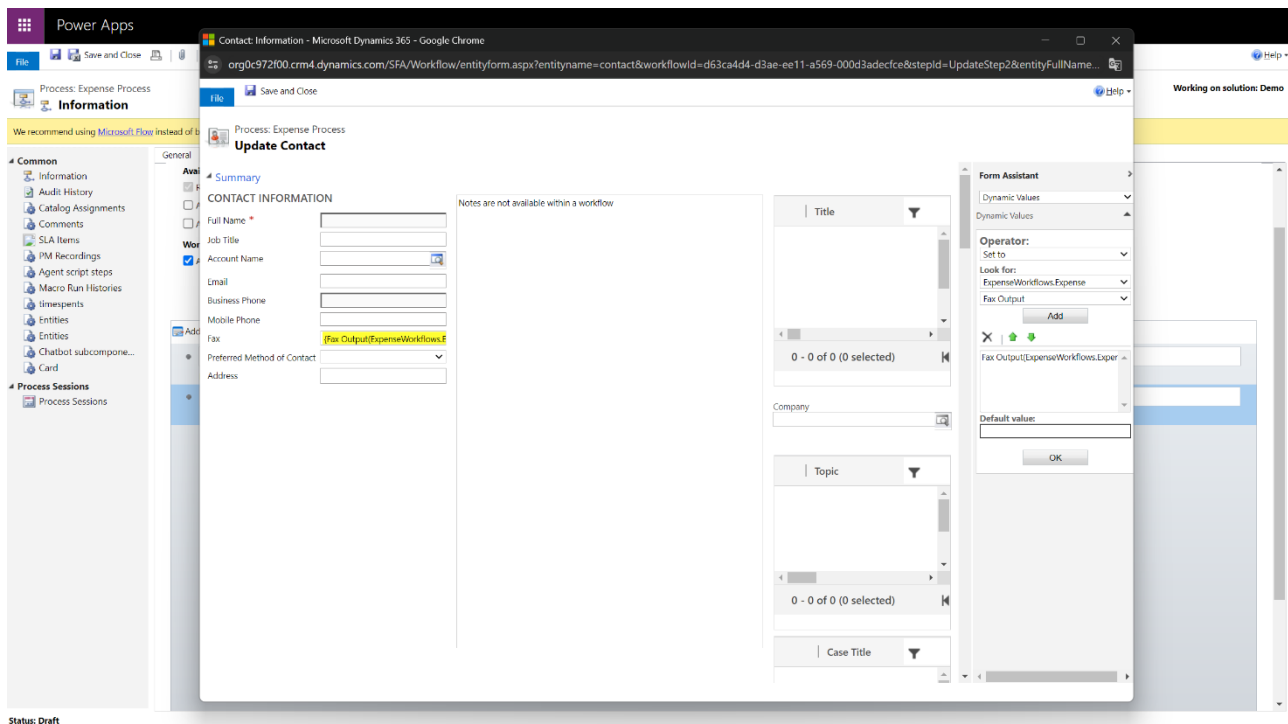


Next, we click on Add Step and choose Update Record. As entity we select Contact and then we click on Set Properties.



Inside Set Properties, in the Look for menu choose ExpenseWorkflows.Expense.

Add Fax Output as value of the Fax field. Then click on Save and Close.



Save the process in the workflow designer and activate it.

At this point, since the custom workflow is running on creation of a new expense, we proceed by creating a new one for Avery Howard (disable the first plugin we created before doing such operation).

If everything worked fine, we should see the Fax field updated with the total amount of expenses associated with Avery Howard.

